

Institute of Information Technology
University of Dhaka



SE 801- Software Project Lab 3

Project Name: Scene Crafter

Supervised By

Dr. Kazi Muheymin-Us-Sakib
Professor
Institute of Information Technology
University of Dhaka

Submitted by

Rufidatul Radium
BSSE 1226

Supervisor's Signature

Date: 29 November , 2024

Letter of Transmittal

12 September, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **Scene Crafter**: A plugin of Godot to help game developers by generating and editing scenes using natural language.

Sir,
I am submitting the technical report for our Software Project Lab 3, "**SceneCrafter:** Automating Scene Setup and Recommending Next Step in Godot".
This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Rufidatul Radium
BSSE-1226
Institute of Information Technology
University of Dhaka

01.12.2024
—----------
Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project "SceneCrafter: Automating Scene Setup and Recommending Next Step in Godot". I would like to convey my tremendous gratitude to my supervisor, Dr. Kazi Muheymin-Us-Sakib, Professor, Institute of Information Technology, University of Dhaka, for providing me with guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

This project focuses on creating a plugin for Godot Engine that allows developers to create and modify game scenes using natural language input. By simply typing a description of what they want. Developers can quickly generate scenes, complete with nodes, properties, and scripts.

The plugin also provides recommendations and suggestions to help the game development process further. It saves time by automating repetitive tasks and makes the engine more accessible to beginners who may not be familiar with Godot's technical aspects.

Introduction **1**
Motivation 1
Scope 2
Intended Audience 2
Purpose of Document 3
Conclusion 3
Understanding the Basic Problem **3**
1. Inception **4**
1.1 Identify the Client of the Project 4
1.2 Icebreaking 4
1.3 Identifying the Stakeholders of the Project 4
1.4 Recognizing Multiple Viewpoints 5
2. Elicitation **5**
2.1 Collaborative Requirements Gathering 6
2.2 Quality Function Deployment (QFD) 6
2.3 Categorization of Requirements 6
2.3.1 Normal Requirements 6
2.3.2 Expected Requirements 6
2.3.3 Exciting Requirements 7
2.4 Usage Scenario 7
3. Scenario Based Modeling **8**
3.1 Use Case Diagram 8
3.1.1 Level-0 9
3.1.2 Level-1 9
Use Cases: 10
3.1.3 Level-1.1 10
3.1.4 Level-1.4 11
4. Data Based Modeling 12
4.1 Introduction 12
4.2 Data Object Identification 12
4.3 Entity Relationship (ER) Diagram 12
5. Class-Based Modeling 14
5.1 Final List of Class 14
5.2 Class Card 14
5.2.1 Scene 15
5.2.2 Node 15
5.2.3 Script 15
5.2.4 Plugin 16
5.2.5 Suggestion 16
5.2.6 Feedback 17
5.2.7 Tracking 17

5.2.8 Natural Language Processor 17
5.3 Summary Table 18
6. Architectural Design **19**
6.1 Architectural Context Diagram(ACD) 19
6.2 Define ArcheTypes 20
6.3 Refining the Architecture into Components 22
6.3.1 Top Level Components 22
6.3.2 Describing Instantiation of the System 22
Preliminary Test Plan **25**
7.1 High Level Description of Testing Goals 25
7.2 Test Cases 25
Test Case 1: Scene Generation from Natural Language 25
Test Case 2: UI Interaction 25
Test Case 3: Performance Under Load 26
Test Case 4: Suggestions Based on Context 26
Test Case 5: Feedback Analysis 27
Test Case 6: Scene Generation from Complex Description 27
Test Case 7: User Interface Performance and Responsiveness 27
Test Case 8: Error Handling in Scene Generation 28
Test Case 9: NLP Parsing Accuracy 28
Test Case 10: User Preferences Adaptation 29
7. Conclusion **29**

List Of Images

- Figure 1. Level 0 - Scene Crafter
- Figure 2. Level 1 - Scene Crafter
- Figure 3. Level 1.1 - Automated Scene Generation
- Figure 4. Level 1.4 - Feedback Mechanism
- Figure 5. ER Diagram
- Figure 6. CRC Diagram
- Figure 7. Architectural Context Diagram(ACD)
- Figure 8. Godot Engine Architecture
- Figure 9. Pipe and Filter Architecture
- Figure 10. Top Level Components of the system
- Figure 11. Instantiation of Scene Processing and Recommendation AI function with component elaboration
- Figure 12. Godot Engine Architecture of PlugIn System
- Figure 13.
- Figure 14. Scene Generation Component
- Figure 15. Feedback Analysis Component

List Of Tables

- Table 1. ER Schema
- Table 2. Scene Class Card
- Table 3. Node Class Card
- Table 4. Script Class Card
- Table 5. Plugin Class Card
- Table 6. Suggestion Class Card
- Table 7. Feedback Mechanism Class Card
- Table 8. Tracking Class Card
- Table 9. Natural Language Processing Class Card
- Table 10. Classes Summary
- Table 11. Test Case 1- Scene Generation from Natural Language
- Table 12. Test Case 2- UI Interaction
- Table 13. Test Case 3- Performance Under Load
- Table 14. Test Case 4- Suggestion based on Contest
- Table 15. Test Case 5- Feedback Analysis
- Table 16. Test Case 6 - Scene Generation from Complex Description
- Table 17. Test Case 7 - Verify the performance and responsiveness
- Table 18. Test Case 8 - Error Handling
- Table 19. Test Case 9 - Accurate parsing
- Table 20. Test Case 10 - Adaption to user preferences

Introduction

SceneCrafter is a plugin designed to help game developers using the Godot Engine create scenes more easily and quickly. In Godot, building scenes involves setting up nodes, configuring properties, and attaching scripts. SceneCrafter is used to simplify this process by allowing developers to describe the scene they want in plain language. The tool then generates the required nodes, sets properties, and attaches scripts automatically. Additionally, SceneCrafter offers real-time recommendations, like reminding users to connect signals or suggesting scripts. This makes creating scenes in Godot smoother, especially for smaller teams or indie developers.

Motivation

In the world of game development, Godot has become a popular choice, particularly among indie developers, hobbyists, and small studios. Its open-source nature, zero licensing fees, and flexibility make it an appealing alternative to engines like Unity and Unreal, which may come with fees or royalties. While Unity and Unreal are powerful, they can be financially burdensome for smaller teams or individuals working on low-budget or free projects. Godot, on the other hand, offers a straightforward and affordable solution with a community-driven development model that encourages accessibility and growth.

However, while Godot is powerful, it still requires users to have a deep understanding of its node-based scene system and scripting requirements. Setting up scenes involves manually creating and configuring nodes, defining properties, and attaching scripts to control behavior. For new or non-technical developers, this process can be tedious, and it often slows down the creative flow. Repetitive tasks, like setting up basic nodes and connecting signals, add to the complexity and time required to build a scene. For experienced developers, these repetitive tasks can interrupt productivity, and for beginners, they create a steep learning curve.

SceneCrafter is designed to reduce the burden of manual scene setup by allowing developers to describe scenes in simple language. It converts these descriptions into the appropriate nodes, properties, and scripts, making the scene creation process much faster and more intuitive. This approach not only speeds up development but also frees developers to focus on the creative aspects of their projects rather than getting bogged down by repetitive tasks.

Without SceneCrafter, developers would continue relying on traditional methods: setting up nodes one by one, configuring each property manually, and ensuring all necessary scripts are attached. This process is manageable for small, simple scenes but quickly becomes overwhelming in larger projects or complex scenes. SceneCrafter's real-time guidance also offers something unique: contextual recommendations that provide prompts to attach missing scripts, configure signals, or fix other potential issues in the scene setup.

Scope

The *Scene Crafter* project focuses on enhancing the Godot game engine with an AI-powered plugin that simplifies scene creation and provides real-time recommendations to developers. The key features and functionalities of this project include:

- **Automated Scene Generation**: Transform natural language descriptions into actionable scenes by creating nodes, configuring properties, and attaching relevant scripts.
- **Contextual Recommendations**: Offer real-time suggestions such as attaching missing scripts, configuring signals, optimizing scene structures, and providing animations or code blocks.
- **Non-Intrusive Integration**: Implement the tool as a Godot Editor plugin, ensuring it works as an add-on without modifying the engine's source code.
- **User-Friendly Interface**: Use Godot's Control nodes to create a custom UI for input and suggestions.
- **Feedback Mechanism**: Monitor user actions to improve suggestions based on observed patterns and common needs.
- **Code Assistance**: Automate script generation using AI to reduce repetitive tasks and improve productivity.
- **Real-Time Monitoring**: Track user interactions for better understanding and support of scene-building activities.

Intended Audience

This SRS is designed for a broad range of stakeholders involved in the development and use of Scene Crafter. It includes:

- **Game Developers**: Independent developers or small teams using the Godot Engine for game development, seeking tools to enhance productivity and simplify scene creation.

- **Technical Leads**: Professionals managing game development teams who want to integrate AI-powered tools to streamline workflows and improve team efficiency.

- **Godot Community Contributors**: Developers and contributors involved in the Godot ecosystem who may explore and extend the plugin's capabilities or use it as a reference for similar projects.

- **Software Engineers**: Engineers interested in AI-powered tools for game development and how they integrate with existing development environments like Godot.

Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

Conclusion

The SceneCrafter project aims to revolutionize the game development process in the Godot Engine by introducing an AI-powered assistant that automates scene creation and provides real-time contextual recommendations. By integrating natural language processing, smart guidance, and non-intrusive plugin design, it addresses key pain points faced by developers, especially in small teams or individual projects. With its focus on usability and adaptability, SceneCrafter represents a step forward in creating intelligent tools for game development.

Understanding the Basic Problem

Game developers often face challenges in creating and managing scenes in the Godot Engine. The process of manually adding nodes, configuring properties, attaching scripts, and setting up signals can be time-consuming and error-prone, especially for complex projects. Developers also lack tools that provide real-time guidance or automation for repetitive tasks.

Currently, no solution exists within Godot to simplify scene creation through natural language input or to suggest contextual actions based on the developer's workflow. This increases the technical barrier for new developers. The absence of an intelligent assistant makes the development process tedious, leaving room for errors and inconsistent setups.

The *Scene Crafter* project addresses these challenges by introducing a plugin that automates scene creation and provides real-time suggestions, thereby assisting developers of all skill levels.

1. Inception

The *SceneCrafter* project was initiated to address the challenges faced by game developers using the Godot Engine, specifically in automating scene creation and providing real-time guidance. Developers often struggle with repetitive tasks like adding nodes, attaching scripts, and configuring signals, which slows down the development process. This project aims to enhance the Godot engine by integrating an AI-powered tool that simplifies scene setup through natural language input and contextual recommendations.

1.1 Identify the Client of the Project

The primary clients for the *SceneCrafter* project are **game developers** using the Godot Engine, especially those working in small teams or solo projects. These clients seek tools that reduce the complexity of scene creation, improve productivity, and help them focus more on creativity rather than repetitive tasks. The secondary clients are technical leads or managers who want to integrate smart tools into their team's workflow to enhance efficiency.

1.2 Icebreaking

To ensure the success of this project, an icebreaking approach is essential in understanding the challenges developers face when using Godot. A collaborative and interactive approach would involve developers providing feedback on pain points and use cases related to scene creation. This step allows me to design a plugin that directly addresses real-world needs, fostering trust and understanding among all participants.

During the initial phase of understanding the project's scope, I had a conversation with Arban, a game developer employed at Softograph, a company that specializes in game development. Arban shared his frustration with the repetitive and error-prone process of scene creation in Godot. After spending numerous hours manually adding nodes, configuring properties, and attaching scripts, he expressed the need for a more efficient system that could automate the scene-building process.

1.3 Identifying the Stakeholders of the Project

- **Game Developers**

  The primary users of the plugin, whose needs drive the project's functionality, such as simplifying scene creation and providing real-time guidance.

- **Technical Leads/Project Managers**

  Responsible for overseeing the integration of the tool into a team's workflow and ensuring it improves productivity and reduces errors.

- **Godot Engine Community**

  As contributors to the Godot open-source engine, this group will play a role in maintaining and evolving the plugin in line with Godot updates.

- **Developers of the Plugin (Project Team)**

  The internal team responsible for designing, coding, testing, and maintaining the plugin, ensuring it meets the defined requirements.

1.4 Recognizing Multiple Viewpoints

- **Developer Viewpoint**

  Developers expect a tool that automates repetitive tasks, reduces errors, and provides suggestions based on their actions. They value efficiency, ease of use, and the ability to customize generated elements to suit their needs.

- **Technical Lead Viewpoint**

  Technical leads focus on integrating the plugin into the team's workflow and ensuring that it helps developers without disrupting current practices.

- **Godot Community Viewpoint**

  As the plugin will be open-source, the Godot community will focus on its extensibility, maintenance, and alignment with Godot's standards. They want to see a well-documented, easy-to-integrate tool that fits within the Godot ecosystem.

- **Project Team Viewpoint**

  The internal team aims to create a user-friendly, effective plugin that addresses the needs of developers while maintaining high standards of performance and integration. They need to balance functionality with usability and keep development on schedule.

2. Elicitation

Elicitation is the process of gathering information and understanding the needs of the stakeholders for the *Scene Crafter* project. To achieve this, multiple methods were employed, including interviews, surveys, and observations. The conversations with game developers like Arban, who expressed challenges in scene creation in Godot, were instrumental in identifying the key features of the tool. These discussions highlighted the pain points in scene building, such as repetitive tasks, manual script generation, and the lack of real-time recommendations.

The goal of the elicitation process was to understand these issues in-depth and gather actionable insights to inform the system's design and features.

2.1 Collaborative Requirements Gathering

Collaborative requirements gathering involved working closely with various stakeholders to ensure that the final system design would meet their needs and expectations. Developers like Arban shared their experiences and expectations for the tool, such as automating scene creation and offering contextual suggestions.

By collaborating with stakeholders, the requirements were refined and prioritized, ensuring the tool would solve the identified problems and enhance developer productivity.

2.2 Quality Function Deployment (QFD)

Quality Function Deployment (QFD) is a method used to translate the needs and desires of stakeholders into specific technical requirements. For *SceneCrafter*, this process involved mapping the features and functionalities that game developers, technical leads, and studios wanted to the technical aspects of the system. It ensured that the tool would meet both functional and non-functional requirements effectively. QFD helped prioritize features based on their importance to stakeholders, translating those into clear, actionable design criteria.

2.3 Categorization of Requirements

2.3.1 Normal Requirements

These are the basic features that are essential for the tool to function properly. These are the expected capabilities, and failure to meet them would lead to dissatisfaction or unusability.

- Ability to generate nodes, configure properties, and attach scripts based on user descriptions.
- Support for generating both 2D and 3D scenes.
- Operates as a plugin without modifying the core Godot engine.
- A simple input field for prompts and easy-to-understand suggestions.
- Automatically create and attach minimal working scripts for generated nodes

2.3.2 Expected Requirements

These are the features that stakeholders expect from the tool to improve their workflows but are not strictly required for the tool’s basic operation. These features would provide significant value, and their absence would cause dissatisfaction.

- Provide contextual recommendations for animating nodes, optimizing scene structure, or configuring signals between nodes.

- Monitor user interactions in real-time to provide relevant feedback and suggestions.
- Allow users to customize or tweak the AI-generated nodes, properties, and scripts easily.

2.3.3 Exciting Requirements

These are additional features that go beyond user expectations and would delight stakeholders, providing a competitive edge to the tool. While these are not essential, they would significantly enhance the user experience.

- Analyze user actions and dynamically adapt suggestions based on observed workflows.
- Provide error messages or warnings if something goes wrong during scene generation

By categorizing the requirements into normal, expected, and exciting, we ensure that the project focuses on delivering the essential features first, then builds up to more advanced features that will provide a superior user experience.

2.4 Usage Scenario

Scene Crafter is a powerful tool designed to accelerate game development in Godot Engine. SceneCrafter automates many of the tedious tasks involved in scene creation, allowing developers to focus on the core game strategy and level design aspects of their projects.

Initialization and Setup:
The user begins by installing the plugin as an add-on in the Godot Engine editor. Once activated, the plugin integrates itself into the editor interface, providing accessible options for natural language input.

Natural Language-Driven Scene Creation:
Developers use the plugin to generate new scenes by entering descriptive commands about their requirements. Based on the input, the system creates a scene hierarchy, configures necessary nodes, and attaches relevant scripts. This eliminates the need for developers to manually configure basic structures, allowing them to focus on refining creative elements.

Real-Time Monitoring and Suggestions:
While the user interacts with the editor, the plugin continuously monitors actions, such as adding nodes, connecting signals, or configuring properties. It identifies potential gaps or inefficiencies in the workflow and suggests appropriate solutions or enhancements. Recommendations are provided non-intrusively, ensuring that the developer retains full control over the design process.

Script Automation and Assistance:
The plugin offers support for generating GDscript code snippets or complete scripts for common functionalities. This includes attaching pre-written templates to nodes or offering

suggestions for missing logic. By providing inline comments and adhering to best practices, the generated scripts serve as a starting point for customization.

User Feedback and Adaptation:
The plugin collects feedback through its real-time monitoring capabilities, observing repetitive tasks or frequently used design patterns. Over time, it adapts to the user's preferences, delivering tailored recommendations and optimizing its outputs to align with the developer's unique style and project requirements.

Enhancement Through Feedback Mechanism:
An exciting feature of the plugin is its ability to provide dynamic feedback to users. By analyzing ongoing projects and user interactions, it highlights potential improvements or offers additional features to elevate the quality of the work. This mechanism fosters continuous learning and development for users while ensuring high standards in project outcomes.

Non-Intrusive Workflow Integration:
The plugin operates entirely as an add-on, refraining from modifying the Godot engine's source code. It respects existing project structures and workflows, ensuring compatibility with user-defined setups and customizations. Developers experience seamless assistance without disruptions to their normal workflows.

Overall, the plugin saves time by automating repetitive tasks, enabling developers to focus on creative aspects of game design. Whether setting up UI, prototyping gameplay mechanics, or generating reusable assets, the plugin integrates into the Godot. It provides an accessible starting point for beginners while offering customization options for experienced developers to tailor it to their specific needs. With this tool, game development becomes faster, more intuitive, and less tedious.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors

(representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 Level-0

Name : Scene Crafter
Primary Actors : Godot Game Developer
Secondary Actors : Large Language Models, Godot Engine, Godot Plugin

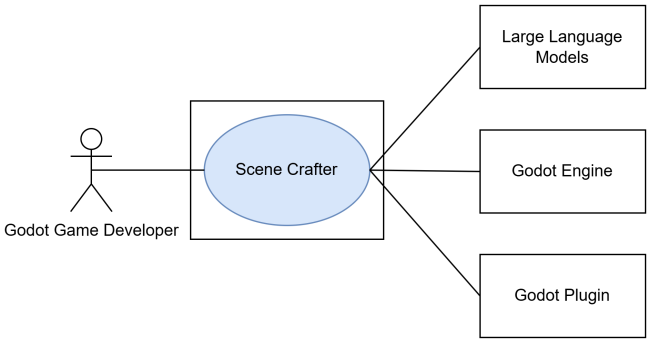


Figure 1. Level 0 - Scene Crafter

3.1.2 Level-1

Name : Scene Crafter
Primary Actors : Godot Game Developer
Secondary Actors : Large Language Models, Godot Engine, Godot Plugin

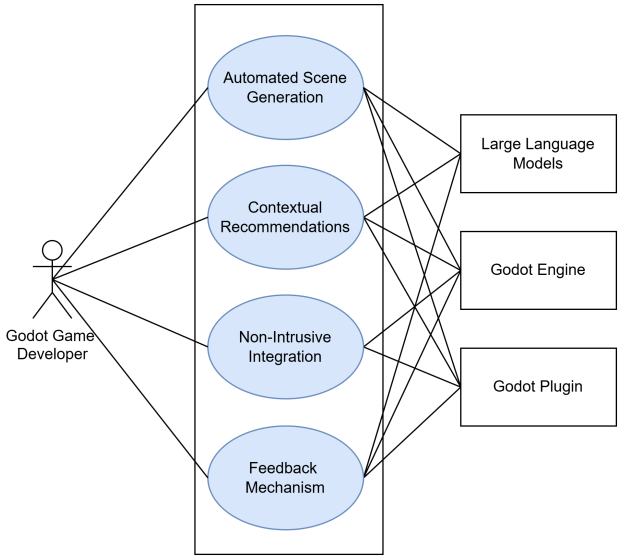


Figure 2. Level 1 - Scene Crafter

Use Cases:

- **Automated Scene Generation**

  Actor: Godot Game Developer, Large Language Model (LLM), Godot Engine and Plugin

  Description: The plugin takes input in natural language, parses the prompt, determines the required nodes and properties, and dynamically creates a new scene file.

  Outcome: The scene file is saved in a certain folder.

- Contextual Recommendation

  Actor: Large Language Model (LLM), Godot Game Developer, Godot Engine and Plugin

  Description: The system suggests relevant enhancements or additional features for scenes and scripts. These recommendations are based on common game development practices, user preferences, or trends in similar projects.

  Outcome: The developer receives suggestions that can improve the scene.

- Non-Intrusive Integration

  Actor: Godot Game Developer, Godot Engine and Plugin

  Description: The plugin seamlessly into the Godot Engine, ensuring that the developer can utilize its features without disrupting their established processes. Options to toggle functionality or customize interaction are provided.

  Outcome: The developer maintains control over their workflow while benefiting from the plugin's capabilities.

- Feedback Mechanism

  Actor: Godot Game Developer, Large Language Models (LLM)

  Description: The system provides feedback based on generated scenes or recommendations. This feedback is used to refine future outputs and improve the model's accuracy and relevance. Feedback mechanisms can include like/dislike, comments, or modifications.

  Outcome: The system evolves over time, aligning closer to user needs and expectations.

3.1.3 Level-1.1

Name : Automated Scene Generation

Primary Actors : Godot Game Developer
Secondary Actors : Large Language Model, Game Engine and Plugin

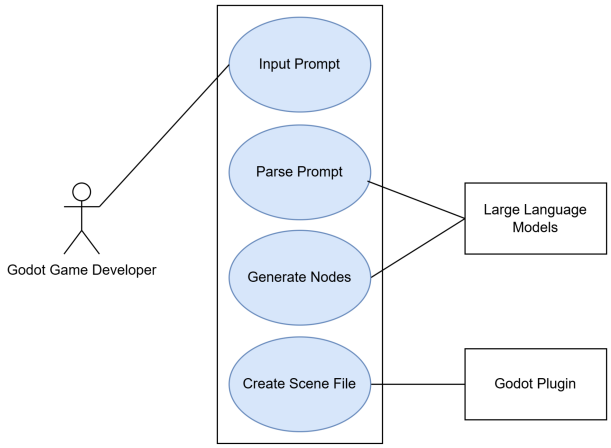


Figure 3. Level 1.1 - Automated Scene Generation

3.1.4 Level-1.4

Name : Feedback Mechanism
Primary Actors : Godot Game Developer
Secondary Actors : Large Language Model, Game Engine and Plugin

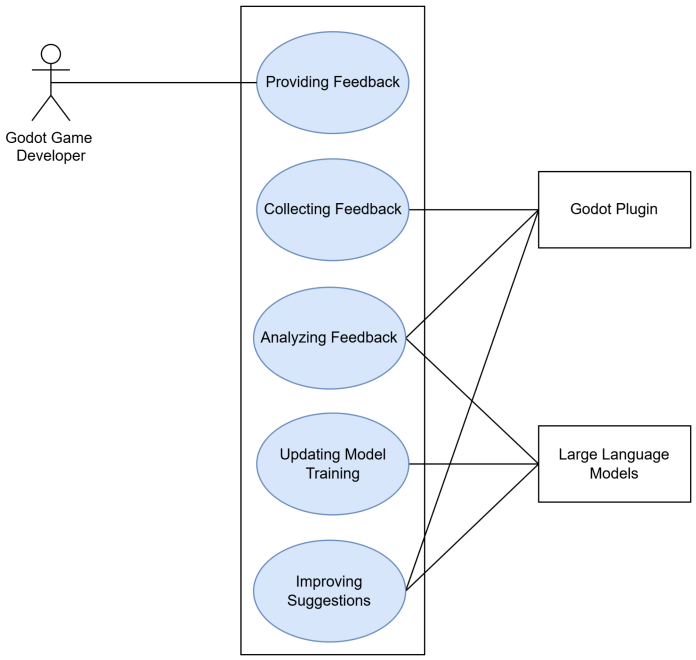


Figure 4. Level 1.4 - Feedback Mechanism

4. Data Based Modeling

4.1 Introduction

A Data Model is an organized view of database concepts and their relationships. The purpose of creating a conceptual data model is to establish entities, their attributes, and relationships. In this data modeling level, there is hardly any detail available on the actual database structure.

The 3 basic elements of Data Modeling are-

- Entity: A real-world thing
- Attribute: Characteristics or properties of an entity
- Relationship: Dependency or association between two entities

The entity relationship diagram (ERD) defines all data objects that are processed within the system, the relationships between the data objects and the information about how the data objects are entered, stored, transformed and produced within the system.

4.2 Data Object Identification

A data object is a collection of one or more data points that create meaning as a whole. In other words, it's a storage or container to store data values. More specifically, data objects are usable, functional, and meaningful artifacts whose form and function is to encode data. It can be used in type checking operations. A data object can be an external entity, a thing, an occurrence, a role, an organizational unit, a place or a structure.

The entities of Scene Crafter after analyzing the usage story are:

- Project
- Prompt
- ChangeOfFile

4.3 Entity Relationship (ER) Diagram

ER Diagram stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships. ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes, triangles to define inheritance and diamond shapes to represent relationships.

The ER-Diagram of Scene Crafter can be shown as below:

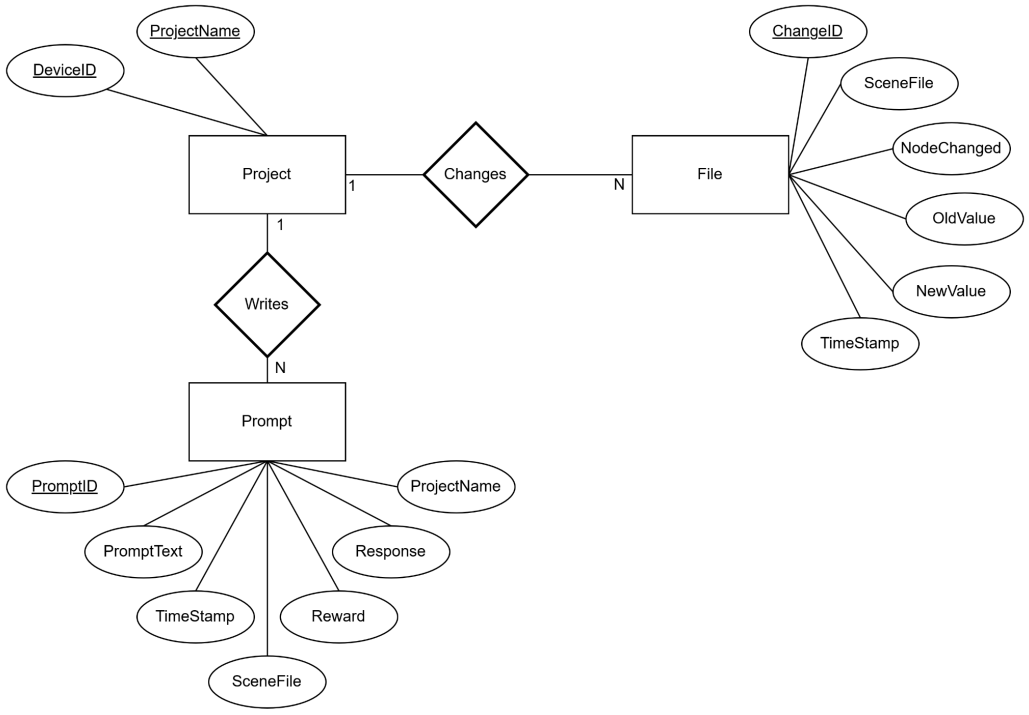


Figure 5. ER Diagram

The schemas derived from the ER-diagram is as follows:

Schema	Attribute	Type
Project	DeviceID	VARCHAR(255) PK
	ProjectName	VARCHAR(255) PK
Prompt	PromptID	SERIAL PRIMARY KEY
	PromptText	TEXT
	TimeStamp	TIMESTAMP
	SceneFile	VARCHAR(255)
	ProjectName	VARCHAR(255) FK
	Response	TEXT
	Reward	int
ChangeOfFile	ChangeID	SERIAL PRIMARY KEY

	SceneFile	VARCHAR(255) FK
	NodeChanged	VARCHAR(255)
	PropertyChanged	VARCHAR(255)
	OldValue	TEXT
	NewValue	TEXT
	TimeStamp	TIMESTAMP

Table 1. ER Schema

5. Class-Based Modeling

By identifying the static structure of the objects in a system, class-based modeling defines the structure of the entire system. A class model defines the attributes and methods of each class's objects, as well as their collaborations across classes within a system. A class-based model has several different components, such as classes and objects, attributes, methods, class-responsibility-collaborator (CRC) models, collaboration diagrams, and packages.

5.1 Final List of Class

I first identified 52 nouns from the scenario. After identifying nouns from the scenario, I filtered those related to the solution domain using general classifications such as external entities, events, roles, organizational units, places, and structures. Potential classes were further refined based on selection criteria like retained information, required services, multiple or common attributes, shared operations, and essential requirements. Through this analysis, I identified the following analysis classes-

- Scene
- Node
- Script
- Plugin
- Suggestion
- Feedback
- Tracking
- Natural Language Processor

5.2 Class Card

The classes of Scene Crafter are given below-

5.2.1 Scene

Class Name: **Scene**	
Attributes	**Methods**
- name - nodes - scripts - properties - layout	+ add_node(node) + remove_node(node) + generate_scene()
Responsibilities	**Collaborator Class**
- Manages scene configuration - Adds/removes nodes and scripts - Generates scene layout	Node, Script, Plugin

Table 2. Scene Class Card

5.2.2 Node

Class Name: Node	
Attributes	Methods
- id - type - position - size - properties - scripts - parent_node - children	+ add_child(child_node) + remove_child(child_node) + attach_script(script) + set_property(property, value)
Responsibilities	Collaborator Class
- Represents a unit in the scene - Can have child nodes - Can have attached scripts and properties	Scene, Script Plugin

Table 3. Node Class Card

5.2.3 Script

Class Name: Script	
Attributes	**Methods**

- language - content - attached_nodes	+ run() + attach_to_node(node) + detach()
Responsibilities	**Collaborator Class**
- Defines behavior of nodes - Executes logic for nodes - Can be attached to or detached from nodes	Node Plugin Scene

Table 4. Script Class Card

5.2.4 Plugin

Class Name :**Plugin**	
Attributes	**Methods**
- name - version - enabled	+ activate() + deactivate() + provide_suggestion()
Responsibilities	**Collaborator Class**
- Extends functionality of the editor - Provides scene generation and suggestions - Activates/deactivates functionality	Scene Feedback

Table 5. Plugin Class Card

5.2.5 Suggestion

Class Name	**Suggestion**
Attributes	**Methods**
- message - type	+ display() + accept() + reject()
Responsibilities	**Collaborator Class**
- Provides recommendations to the developer	Plugin

- Displays suggestions and handles user responses	

Table 6. Suggestion Class Card

5.2.6 Feedback

Class Name: Feedback Mechanism	
Attributes	Methods
- feedback_data - interaction_count - user_preferences	+ track_interaction() + adapt_suggestions() + store_preferences()
Responsibilities	Collaborator Class
- Gathers and processes user feedback - Adapts suggestions based on interaction patterns	Tracking, Suggestion Plugin

Table 7. Feedback Mechanism Class Card

5.2.7 Tracking

Class Name: **Tracking**	
Attributes	**Methods**
- action_type - timestamp - action_context	+ record_action(action) + generate_report()
Responsibilities	**Collaborator Class**
- Monitors user actions in real time - Observes patterns and records actions	Feedback, Plugin

Table 8. Tracking Class Card

5.2.8 Natural Language Processor

Class Name: **Natural Language Processor**	
Attributes	**Methods**
- input_description - parsed_output	+ process_input(description) + generate_scene()

Responsibilities	**Collaborator Class**
- Processes natural language input - Converts descriptions into scene configurations	Plugin Scene

Table 9. Natural Language Processing Class Card

5.3 Summary Table

Class Name	Collaborator Class
Scene	Node, Script, Plugin
Node	Scene, Script, Plugin
Script	Node, Scene, Plugin
Plugin	Scene, Feedback
Suggestion	Plugin
Feedback	Tracking, Suggestion
Tracking	Feedback, Plugin
Natural Language Processor	Plugin, Scene

Table 10. Classes Summary

The CRC diagram of the classes is given below:

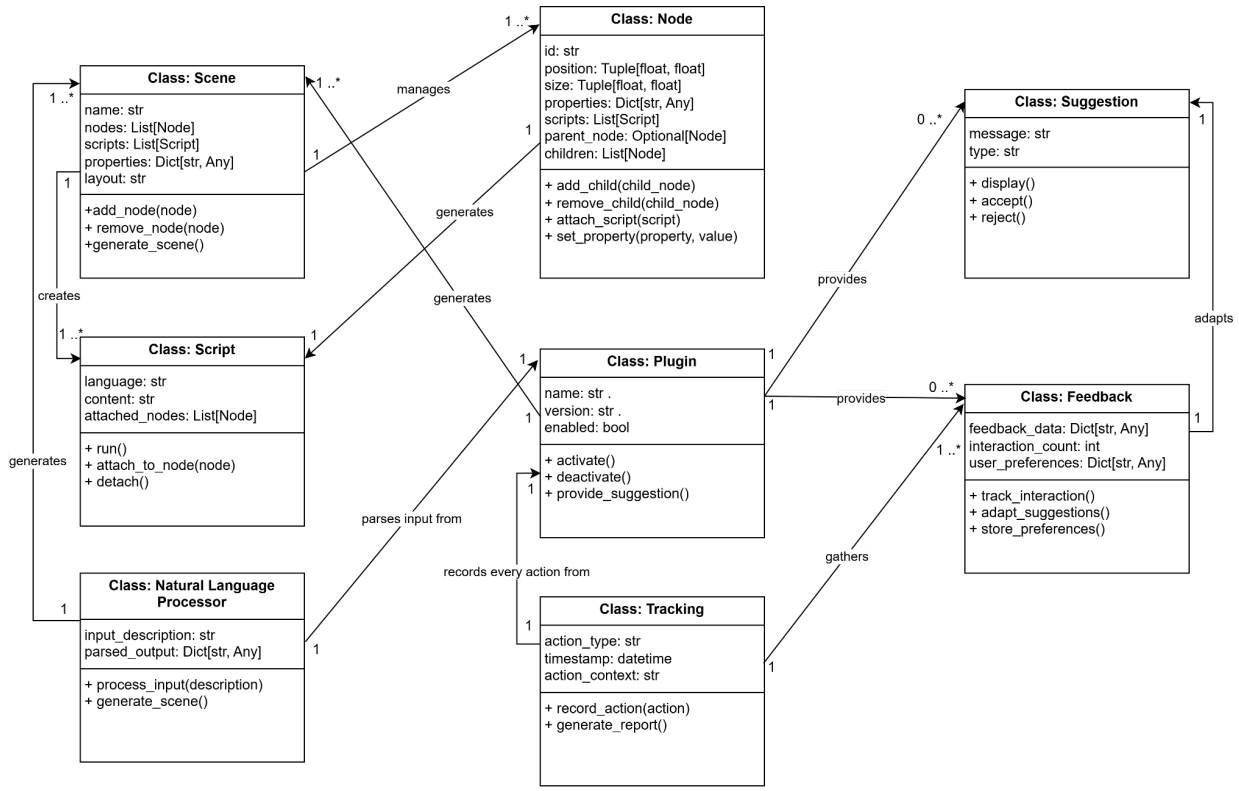


Figure 6. CRC Diagram

6. Architectural Design

As architectural design begins, the software to be developed must be put into context—that is, the design should define what external entities the software interacts with and the nature of the interactions. This information can generally be acquired from the requirements model and all other information gathered during requirements engineering. Software architectural design represents the structure of the data and program components that are required to build a computer-based system. There are 4 steps in architectural design.

6.1 Architectural Context Diagram(ACD)

At the architectural design level, an architectural context diagram (ACD) models how software interacts with external entities, including superordinate, subordinate, and peer-level systems, and actors, all communicating with the target system through interfaces.

Representation of SceneCrafter in Architectural Context Diagram(ACD) :
Superordinate systems - Godot Engine
Subordinate systems - Natural Language Processing (LLM, NLTK)
Peer-level systems - Our system does not have any peer-level system.
Actors - Godot Game Developers

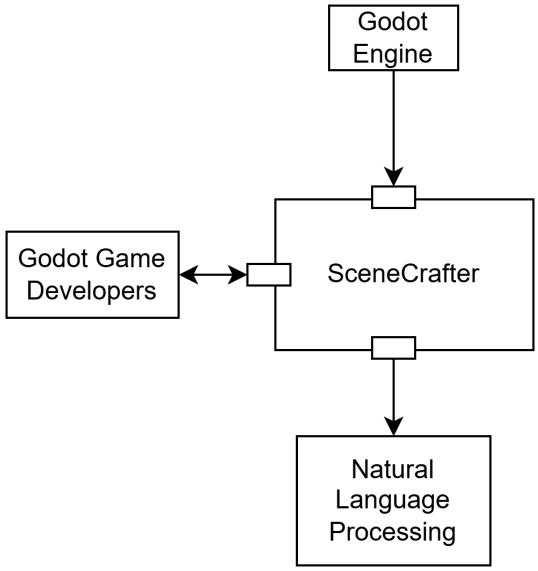


Figure 7. Architectural Context Diagram(ACD)

This classification helps define the relationships between your system and its environment, and how interactions flow between these systems.

6.2 Define ArcheTypes

An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, relatively small set archetypes are required to design even relatively complex systems. The target system architecture is composed of these archetypes, which represent stable elements of the architecture but may be instantiated in many different ways based on the behavior of the system. In many cases, archetypes can be derived by examining the analysis classes defined as part of the requirements model. In our cases, the archetypes are:

1. Godot Engine Architecture
2. Pipe and Filter Architecture

Godot Engine Architecture

Godot embraces object-oriented design at its core with its flexible scene system and Node hierarchy. It tries to stay away from strict programming patterns to offer an intuitive way to structure games.

This architecture allows for flexibility and reusability in game development. Here's a brief overview:

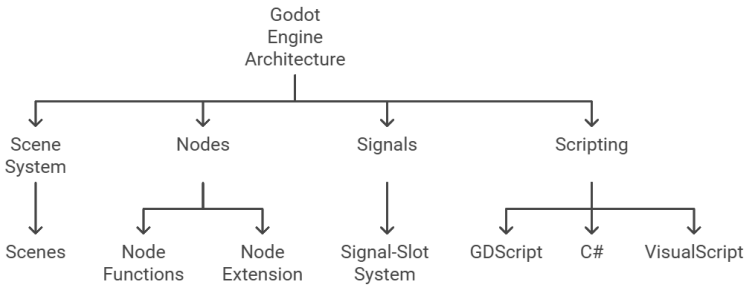


Figure 8. Godot Engine Architecture

Scene System: In Godot, everything is a scene. A scene can be a game level, a character, or even a single object. Scenes can be composed of other scenes, making it easy to manage complex hierarchies.

Nodes: Scenes are made up of nodes. Each node serves a specific purpose, such as displaying graphics, handling physics, or playing sounds. Nodes can be extended or inherited to create custom behaviors.

Signals: Godot uses a signal-slot system for communication between nodes. Signals allow nodes to notify each other when events occur, promoting a decoupled design.

Scripting: Godot uses GDScript, a Python-like scripting language, for extending node functionality. It also supports C# and VisualScript for those who prefer different scripting languages.

This design enables a clear separation of concerns, easy reuse of components, and a flexible structure that can adapt to various game development needs.

Pipe and Filter Architecture

The Pipe and Filter architecture is a design pattern that divides a process into a series of distinct steps, called filters, linked by channels known as pipes. Each filter is dedicated to a particular processing function, whether it involves transforming, validating, or aggregating data. Data flows through these filters via pipes, which carry the output of one filter to the input of another.

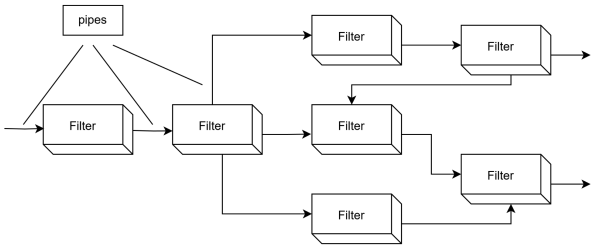


Figure 9. Pipe and Filter Architecture

Filters: Each filter carries out a specific, standalone task, whether it be transforming, validating, or processing data before forwarding it. In the setup, there are two levels of filters; the first one processes data from the pump and hands it to the second filter, which continues the processing before passing it on.
Pipes: Pipes serve as the channels for data movement between filters, linking each component in sequence and ensuring a smooth transfer of data. In diagrams, pipes are depicted as arrows connecting the components.

6.3 Refining the Architecture into Components

6.3.1 Top Level Components

Overall architectural structure with top level components is illustrated below. Note that, Top Level Components diagrams show a very high level view of our system. It is further refined into architecture in the following sections. We keep the top level diagrams to 2 independent responsibilities due to the inherent smaller size of the system and for ease of understanding the system at higher level.

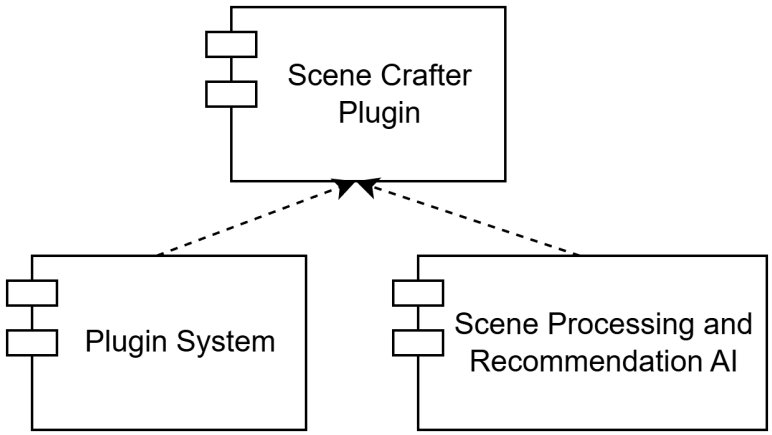


Figure 10. Top Level Components of the system

6.3.2 Describing Instantiation of the System

The components identified in Top Level Components are broad. They need to be refined into smaller components. And just a collection of components doesn't create a meaningful design. So we apply architecture on the components to make a maintainable collection of components. The architecture is applied to a specific problem with the intent of demonstrating that the structure and components are appropriate.

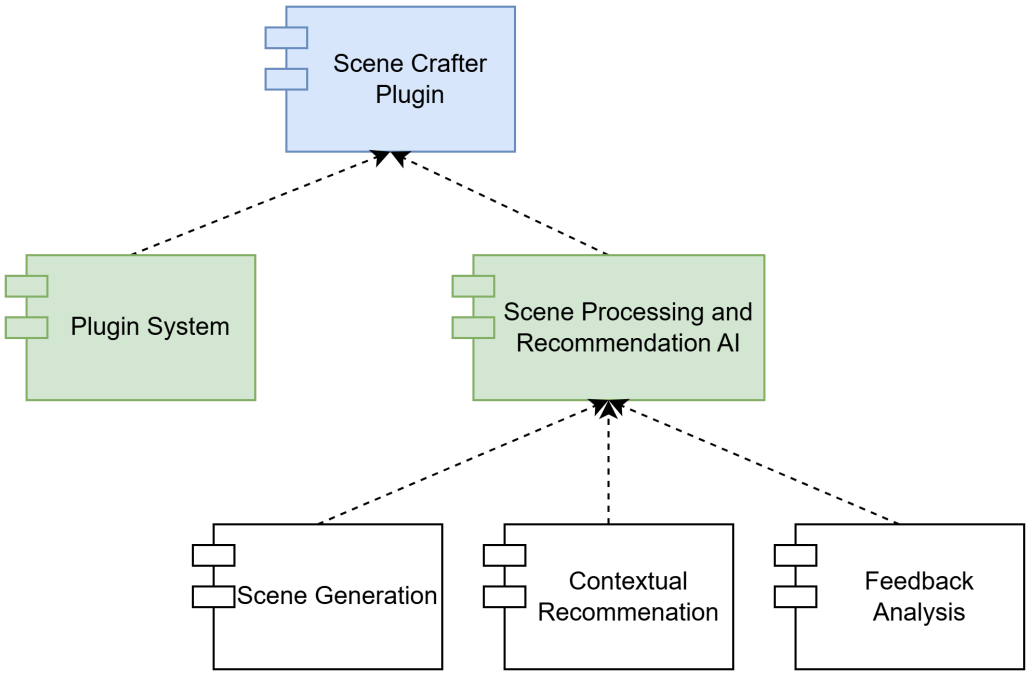


Figure 11. Instantiation of Scene Processing and Recommendation AI function with component elaboration

Component: Plugin System
Pattern: Godot Engine Architecture

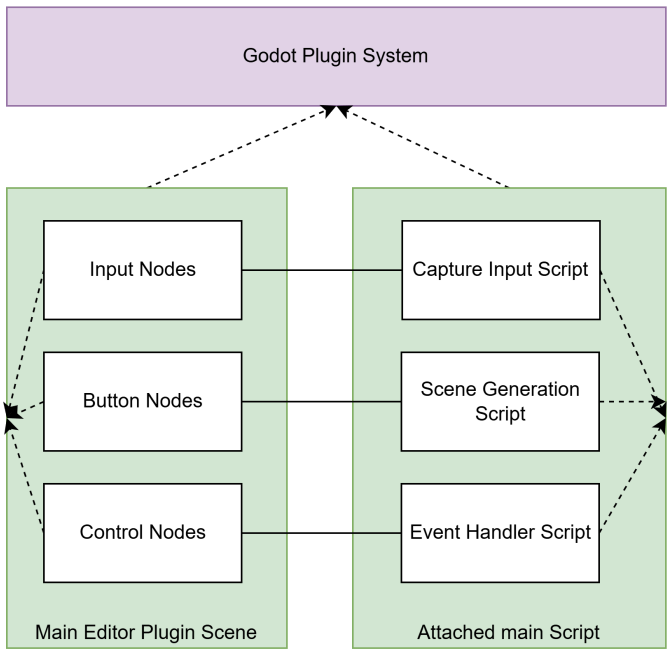


Figure 12. Godot Engine Architecture of PlugIn System

Component: Contextual Recommendation

Pattern: Pipe and Filter

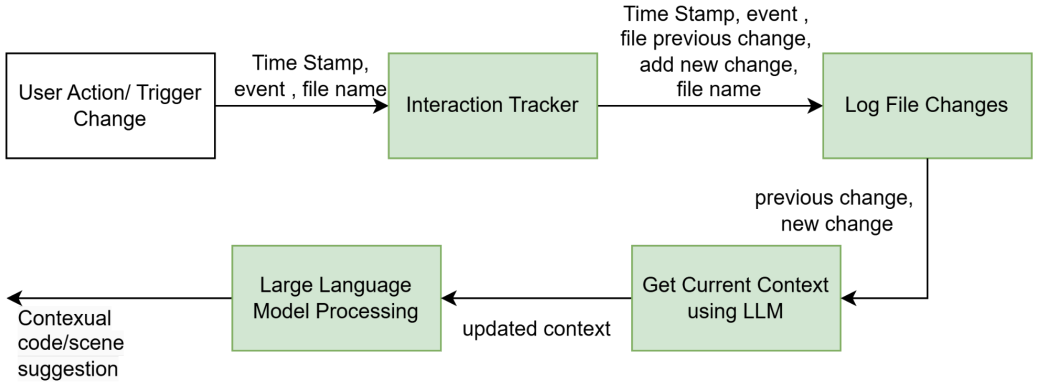


Figure 13.

Figure 14. Pipe and Filter of Contextual Recommendation

Component: Scene Generation

Pattern: Pipe and Filter Architecture

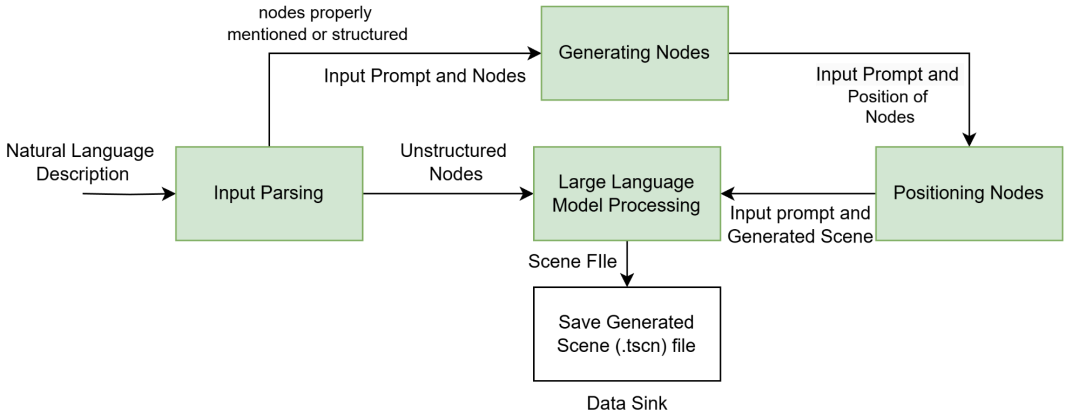


Figure 14. Scene Generation Component

Component: Feedback Analysis

Pattern: Pipe and Filter Architecture

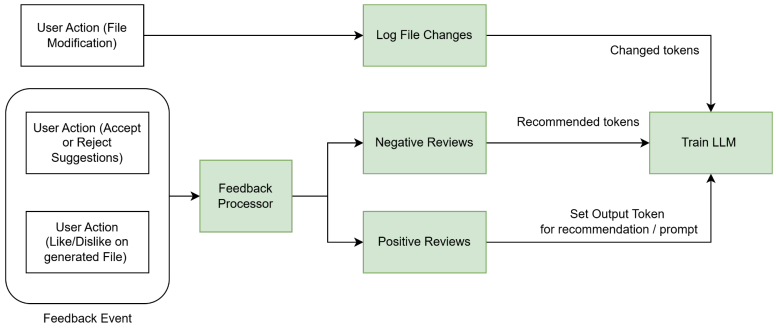


Figure 15. Feedback Analysis Component

Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- To ensure Scene Crafter plugin integrates seamlessly with the Godot Engine
- To validate core functionalities
- To validate that the system accurately interprets user prompts
- To validate that the system generates correct scene structures
- To test the reliability and usability of contextual recommendations and feedback mechanisms
- To confirm the effectiveness of AI model updates based on user feedback.

7.2 Test Cases

Test cases for the testing of this tool are documented below-

Test Case 1: Scene Generation from Natural Language

Test Case ID	TC_SC_01
Objective	Verify that Scene Generation Automation works correctly from natural language input.
Pre-Conditions	The SceneCrafter plugin is installed and enabled in Godot. The system is connected to the LLM model interface.
Test Steps	1. Enter a natural language description for a scene (e.g., "Create a 2D platformer scene with a player character and ground"). 2. Verify that the scene is generated with nodes, properties, and scripts as expected.
Expected Result	A scene is generated with a player character, ground node, and appropriate scripts.

Table 11. Test Case 1- Scene Generation from Natural Language

Test Case 2: UI Interaction

Test Case ID	TC_UI_01

Objective	Verify the UI components work correctly and allow the user to interact with the system.
Pre-Conditions	The SceneCrafter plugin is installed and enabled in Godot.
Test Steps	1. Click on the button to generate a scene from natural language. 2. Verify that the UI updates to show feedback or error messages.
Expected Result	The UI should show progress or feedback messages, and allow further interaction.

Table 12. Test Case 2- UI Interaction

Test Case 3: Performance Under Load

Test Case ID	TC_PT_01
Objective	Verify that the system performs well under heavy load (e.g., large scenes, multiple interactions).
Pre-Conditions	The system is connected to a test environment with heavy workloads (e.g., large scenes).
Test Steps	1. Generate a complex scene with many nodes and scripts. 2. Monitor system performance (e.g., response time, memory usage).
Expected Result	The system should respond within acceptable performance limits

Table 13. Test Case 3- Performance Under Load

Test Case 4: Suggestions Based on Context

Test Case ID	TC_SBC_01
Objective	Ensure that contextual suggestions are appropriate and related to the user's actions in the Godot editor.
Pre-Conditions	The SceneCrafter plugin is installed and the user is interacting with the editor (e.g., adding a node).
Test Steps	1. Perform actions such as adding a new node or a script. 2. Verify if suggestions like "Attach script to node" or "Configure node properties" are displayed.

Expected Result	Relevant suggestions are displayed based on the user's interaction with the editor.

Table 14. Test Case 4- Suggestion based on Contest

Test Case 5: Feedback Analysis

Test Case ID	TC_FA_01
Objective	Test if the feedback tracks and adapts suggestions based on user actions over time.
Pre-Conditions	The system is tracking user interactions and providing suggestions.
Test Steps	1. Repeatedly perform actions like adding specific types of nodes (e.g., RigidBody2D) and attaching scripts. Check if the system begins recommending relevant actions tailored to the user's habits (e.g., attaching a script after adding a RigidBody2D)
Expected Result	The feedback system adapts to repeated actions and offers improved suggestions over time.

Table 15. Test Case 5- Feedback Analysis

Test Case 6: Scene Generation from Complex Description

Test Case ID	TC_SC_02
Objective	Ensure that the system can handle more complex natural language descriptions and generate the corresponding scene.
Pre-Conditions	The SceneCrafter plugin is installed and connected to the NLP system.
Test Steps	1. Provide a complex scene description such as "Create a 3D scene with a moving camera, a player character with basic physics, and background music.". 2. Verify that the system generates a scene with the correct nodes (e.g., Camera, Character, Audio) and scripts for physics and movement.
Expected Result	The scene should be correctly generated with all nodes and scripts necessary to fulfill the complex description.

Table 16. Test Case 6 - Scene Generation from Complex Description

Test Case 7: User Interface Performance and Responsiveness

Test Case ID	TC_UI_02
Objective	Verify the performance and responsiveness of the UI under heavy usage.
Pre-Conditions	The SceneCrafter plugin is active and the user is performing multiple actions in the editor.
Test Steps	1. Continuously interact with the UI by adding and removing nodes, attaching scripts, and requesting suggestions. 2. Monitor if the UI remains responsive without noticeable delays or freezing.
Expected Result	The UI should remain responsive even under heavy usage and provide timely feedback.

Table 17. Test Case 7 - Verify the performance and responsiveness

Test Case 8: Error Handling in Scene Generation

Test Case ID	TC_EH_01
Objective	Ensure the system gracefully handles errors during scene generation from natural language.
Pre-Conditions	The SceneCrafter plugin is active.
Test Steps	1. Provide an incomplete or invalid natural language description (e.g., "Create a scene with no objects"). 2. Observe if the system generates an appropriate error message or feedback.
Expected Result	The system should show an error message or handle the issue gracefully without crashing.

Table 18. Test Case 8 - Error Handling

Test Case 9: NLP Parsing

Test Case ID	TC_NLP_01
Objective	Test the NLP component's ability to accurately parse and convert natural language descriptions into scenes.
Pre-Conditions	The SceneCrafter plugin is connected to the NLP system.

Test Steps	Provide different natural language descriptions with varying levels of complexity.
Expected Result	The NLP system should parse and convert natural language input into correct scene configurations.

Table 19. Test Case 9 - Accurate parsing

Test Case 10: User Preferences Adaptation

Test Case ID	TC_UP_01
Objective	Ensure the feedback and suggestions adapt to user preferences over time.
Pre-Conditions	The SceneCrafter plugin is installed, and the system is tracking user actions.
Test Steps	1. Perform actions like creating specific types of nodes or scripts repeatedly. 2. Ensure that the system starts adapting the suggestions based on user habits and preferences (e.g., suggesting frequently used node types).
Expected Result	The suggestions should adapt to the user's habits and preferences, offering more relevant suggestions over time.

Table 20. Test Case 10 - Adaption to user preferences

7. Conclusion

The **Scene Crafter** plugin represents a significant leap forward in game development efficiency within the Godot Engine. By automating many of the tedious and repetitive tasks involved in scene creation, it empowers developers to focus more on the creative and strategic aspects of their projects. Through the integration of natural language processing, real-time monitoring, and adaptive feedback mechanisms, this tool not only simplifies the development process but also enhances the overall user experience.